

- There are many cases other than function calling where python code gets executed e.g. on attribute access, automatic coercion to bool in `if/else` if statements and automatic coercion to iter in `for` statement. So I'd to cover them up too.
- The `call_handler` was making whole function body unhygienic because any other usage of name `call_handler` will clash with it. So we thought to make a namespace with a very unique name that nobody will define in any other place. Here, we came up with word `goofy` and replaced decorator `profile` with `goofy.profile()` and replaced `call_handler` with `goofy.call_handler`. And the only thing that we'll have to import is `goofy`.
- In `goofy.call_handler` we need to pass other information like `lineno`, `colno` too along with the calling function.

Here is the code which transforms the AST

```
import ast
class Transformer(ast.NodeTransformer):
    def __init__(self, lineno):
```

`lineno` is the actual position of function code in source file

```
        self._lineno = lineno
        self._dec_lineno = 1
    def visit_FunctionDef(self, func):
```

Remove decorators so that the decorators doesn't get applied more.

```
        in func.decorator_list:
            if (isinstance(dec, ast.
Call) and isinstance(dec.func,
ast.Attribute) and
isinstance(dec.func.value,
ast.Name) and dec.
func.value.id == 'goofy' and
dec.func.attr == 'profile'):
                self._dec_lineno = dec.lineno
            func.decorator_list = []
        func_ast = self.generic_visit(func)
        return func_ast
```

```
    def visit_Call(self, call):
```

Change the function calling syntax `func(*args, **kwargs)` will be transformed `goofy.call_handler(func, line_no, col_no, *args, **kwargs)`

*pointers

>>Interesting Django and other developer videos



*Django for Beginners

>>A comprehensive Django tutorial for beginners or those looking to brush it up.

<http://dgit.in/DjngFrBgnsr>



*Why use Django

>>Should you choose Ruby on Rails vs Django? Node.js vs Django? PHP vs Python?

<http://dgit.in/DjngoVS>



*Google Cast SDK, Version 3, for Android

>>New cast SDK reduces the amount of code needed to Cast-enable your Android app.

<http://dgit.in/CastSDK3>



*CodeJam Finals 2016

>>The CodeJam 2016 Finals, where Gennady Korotkevich defended his title against 25 top programmers.

<http://dgit.in/CdJm16>



```
        call_ast = self.generic_
visit(call)
        call_ast.args.
insert(0, call_ast.func)
        call_ast.func =
ast.Attribute(
            value=ast.Name(id='goofy',
ctx=ast.Load()),
            attr='call_handler', ctx=ast.Load())
        call_lineno = self._lineno -
self._dec_lineno + call_ast.lineno
        call_colno = call_ast.col_offset + 1
        call_ast.args.insert(1, ast.
Num(call_lineno))
        call_ast.args.insert(2, ast.
Num(call_colno))
        return call_ast
```

There are many variants of statements and expressions in python, details of which you can find in AST docs's Abstract grammar section. So any type of node that has to be changed while transforming the AST, a corresponding method of `visit_<NodeClass>` in `Transformer` class has to be written. (like `visit_FunctionDef` and `visit_Call` methods in above code). In actual code, I'd to implement `visit_Assign`, `visit_Attribute`, `visit_If`, `visit_BoolOp` and `visit_For` etc. too.

And here's the code for `goofy` class.

```
def goofy_profiler(f):
    frame, filename, line_number,
    _, lines, _ = inspect.stack()[1]
    if lines[0].startswith('def'):
        line_number -= 1
    source = inspect.getsource(f)
    decorator_lineno = source.count('\n',
0, source.index('@goofy.profile')+1) + 1
    tree = ast.parse(source)

    transformer = Transformer(line_number)
    transformed_tree = trans-
former.visit(tree)

    ast.fix_missing_
locations(transformed_tree)
    ast.increment_lineno(transformed_tree,
line_number - decorator_lineno)

    module_globals = inspect.
getmodule(f).__dict__
    exec(compile(transformed_tree,
filename=filename, mode='exec'),
module_globals)
    func = eval(f._name_, module_globals)
    func = deco(func)
    return func
```